

# 调度序驱动的驻留构成优化：面向 NPU 的溢出代价感知方法

高成志 黄骏 叶勤

contact@vennai.org hj992881627@outlook.com yq020319@163.com

## 摘要

深度学习编译器在将算子编译为核函数时，会产生由微操作、短生命周期张量和异构流水线组成的有向无环图。由于片上缓存容量有限，不同的合法调度顺序会导致差异巨大的内存峰值和数据溢出流量。本文揭示了现有调度算法常忽略的一个结构性非对称特征：从主存读入且已有备份的“干净”数据在被驱逐时无需写回，而计算产生的“脏”数据则必须写回，两者的溢出代价截然不同。基于此，本文提出一种溢出代价感知的活跃度（Liveness）塑形方法，通过主动调整容量受限时的干净与脏数据驻留比例，优先驱逐低代价数据，从而保留片上缓存储备。理论上，本文给出单侧的溢出必然性证书，用于判定溢出在近最优调度中是否不可避免，并建立了溢出面积与额外访存流量间的有条件常数近似关系。基于公开 NPU 风格调度实例、合成分布、小图 oracle 与受控消融的实验表明，clean/dirty 无感的内存压力调度器在容量受限区域会多付  $2.4\sim 26\times$  的 P2 溢出流量。

**关键词：**核内调度；神经网络处理器；DAG 调度；片上驻留优化；溢出代价；编译器优化

## 1 引言

NPU 核内调度将一个算子级计算图中的数百至数万个微操作映射到有限的片上缓存层次上，这一过程延续了深度学习编译器中图级优化与多层 IR lowering 的基本路径 [4, 10]。调度序必须满足数据依赖，并在片上缓存（scratchpad）的容量约束下管理中间值的驻留与换出。当同时驻留的数据量超出容量时，编译器需要将部分缓冲临时换出到片外内存并在后续重载，这类溢出既增加额外数据搬运，又可能阻塞执行流水线。

已有核内调度方法围绕关键路径、资源压力或峰值驻留量构造拓扑序优先级，将溢出视作容量不足后的局部后果。这一视角遗漏了一个结构性问题：当容量超限不可避免时，不同的合法调度序会在溢出窗口内暴露完全不同的缓冲构成，从而产生截然不同的换出代价。

具体而言，NPU 核内缓冲可自然区分为干净与脏两类：由片外读入且已有备份的干净缓冲在换出时无需写回，代价仅为重载；由计算产生的脏缓冲若后续仍被使用，则必须先写后读，代价恰为前者的两倍。二者在容量模型中占用相同的片上空间，却对应  $2\times$  的溢出代价差异。若调度器只追求降低峰值或只在固定序上调整驱逐规则，就无法利用这一非对称结构。本文将这一通过改变合法拓扑序主动控制溢出窗口内 clean/dirty 构成的自由度称为溢出代价感知的活跃度塑形（spill-cost-aware liveness shaping）。

本文的贡献如下：

1. 溢出代价感知的活跃度塑形框架：通过调度序控制溢出窗口内的 clean/dirty 驻留构成，使高压区保留低成本可换出的缓冲储备。
2. 溢出的理论刻画：溢出必然性定理（Theorem 1）给出线性时间可验证的单侧诊断条件，用于识别给定缓存容量下溢出是否为任何近最优调度都无法避免；Belady-margin 稳定性命题（Proposition 1）表明调度序而非驱逐策略是主要优化自由度；溢出面积近似定理（Theorem 2）在有界缺席时长下建立  $A(S)$  与最优溢出流量的双边常数界，解释代理目标在容量受限区域为何有效。
3. 在公开 NPU 风格基准实例、合成分布、小图 oracle 和受控消融四个层面的系统实验，验证了本方法在缓存容量受限区域的有效性。

实验表明，clean/dirty 无感的内存压力调度器会多付  $2.4\sim 26\times$  的额外溢出流量，且收益集中在定理判定为容量受限的区域。

## 2 相关工作

面向 DNN 加速器的联合调度与内存优化中，COSMA [11] 明确指出额外片外访问取决于 operator schedule、memory allocation 和 tensor replacement 三者的交互，并用 ILP 同时优化以减少 non-compulsory

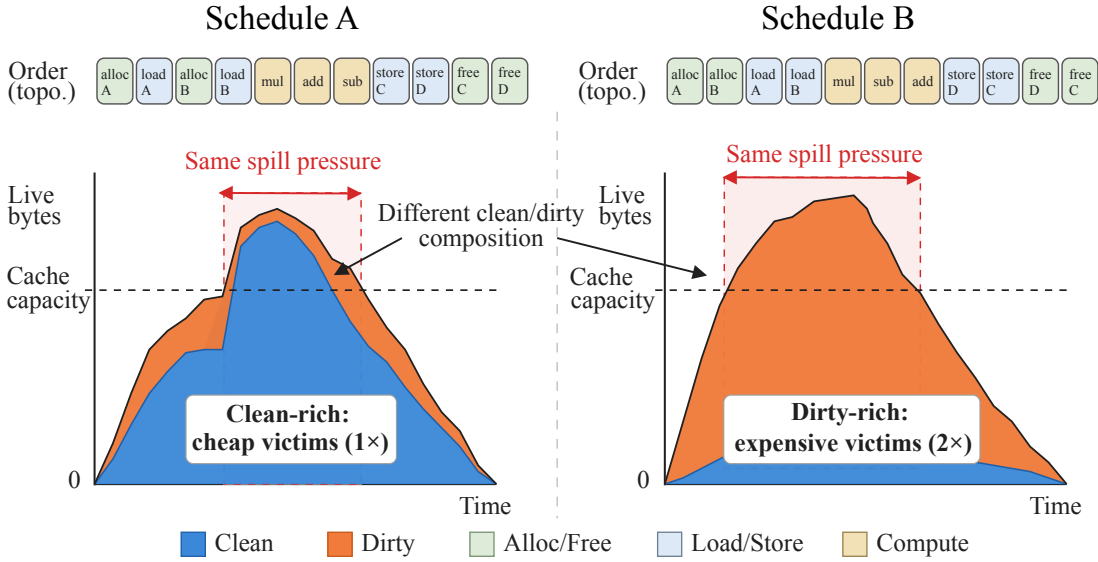


图 1: 溢出代价感知的活跃度塑形概念图。两个合法调度序可能具有相近的容量压力, 但在溢出窗口内暴露给驱逐过程的 clean/dirty 构成不同。当 clean 缓冲在高压区保持常驻时, 同等大小的换出可避免写回, 从而降低额外搬运。

off-chip accesses。该工作在图级算子粒度上统一求解放置与替换, 而核内微操作 DAG 的片上 scratchpad 调度面临更细粒度的约束: 缓冲的 clean/dirty 属性带来  $2\times$  的溢出代价非对称, 调度序不仅影响驻留量峰值, 还决定溢出窗口内可驱逐对象的代价构成。

经典编译器后端早已认识到调度与存活区间的耦合。Goodman-Hsu 风格的 integrated prepass [6] 及后续寄存器压力感知调度 [13, 14] 在指令级并行性与 register pressure 之间折中; 局部分配中的 Clean-First 启发式利用了 clean value 换出无需 store 的事实 [7, 12]。缓存替换理论中, Belady 最远未来原则 [1] 及 block-aware caching [5] 将问题推广到非均匀代价对象。这些工作说明”调度序影响 liveness”和”clean victim 更便宜”并非新概念, 但它们或在固定指令流上选择 victim, 或面对给定请求序列; 核内 DAG 调度的额外自由度在于可通过改变拓扑序主动塑形存活重叠与驻留构成。

重物化与激活检查点从另一侧处理内存压力 [3, 8]; 近期 spill-free compilation [2] 通过缩短活跃范围追求完全消除溢出。这些方法与本文正交: 在真实大模型核内调度中容量约束常使溢出不可避免, 此时已有的可重载输入缓冲构成天然的 clean 储备, 关键在于通过调度序降低不可避免溢出的写回成本。

### 3 问题模型

本文考虑的 DAG 结构与 cache 层级参数来源于公开 NPU 核内调度基准, 该基准模拟编译器生成的核函数级调度问题。输入为一个神经算子的微操作计算图  $G = (V, E)$ , 节点分为两类 (图 2): 执行计算或数据搬运的**操作节点**, 具有执行单元、延迟以及所读写的缓冲集合; 以及标记缓冲生命周期起止的**缓存管理节点**。每个逻辑缓冲  $b$  有唯一的分配节点  $a_b$  与释放节点  $f_b$ , 携带缓冲大小  $s_b$  与所属 cache 类型  $\tau(b) \in \mathcal{C}$ 。每种 cache  $c \in \mathcal{C}$  具有抽象容量  $\text{Cap}_c$  (具体参数见 §5)。一个**调度**  $S = (v_1, \dots, v_m)$  是所有必需节点的一个拓扑序。

称缓冲  $b$  为 clean (干净), 当且仅当它由某个从外存读入数据的操作节点写入——此时片外存储中存在其备份, 溢出换出时无需写回, 额外流量仅为重载代价  $e_b = s_b$ ; 否则称  $b$  为 dirty (脏), 换出必须先写后读, 额外流量  $e_b = 2s_b$ 。这一非对称结构是本文方法的核心出发点: clean 与 dirty 缓冲在容量模型中占用相同的片上空间, 但溢出代价相差  $2\times$ 。

同一代价模型从三个递进视角度量调度质量: P1 (峰值驻留) 只评判节点序, 最小化逐 cache 峰值; P2 (溢出流量) 进一步输出物理地址与溢出列表, 最小化总额外流量  $E(S) = \sum_{b \in \text{Spills}} e_b$ ; P3 (执行时间) 在固定序下按依赖与管线约束求最早完成时间。三者共享上述非对称代价, 因此一个能在溢出窗口保留更

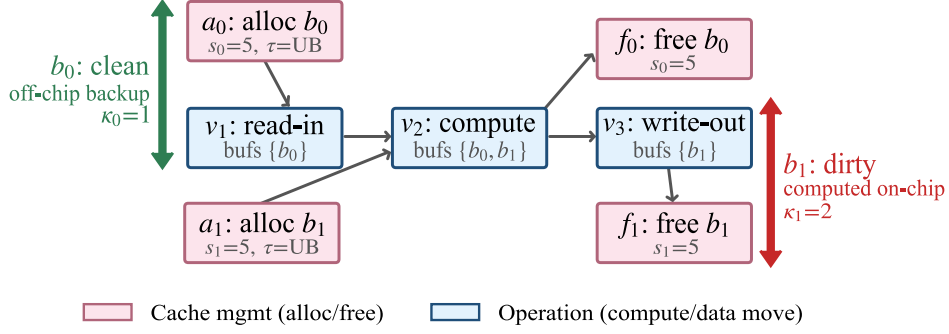


图 2: 微操作 DAG 示例。粉色为缓存管理节点（分配/释放），蓝色为操作节点。缓冲  $b_0$  经外存读入（片外已有备份），属于 clean 缓冲（ $\kappa = 1$ ）；缓冲  $b_1$  由计算产生，属于 dirty 缓冲（ $\kappa = 2$ ）。

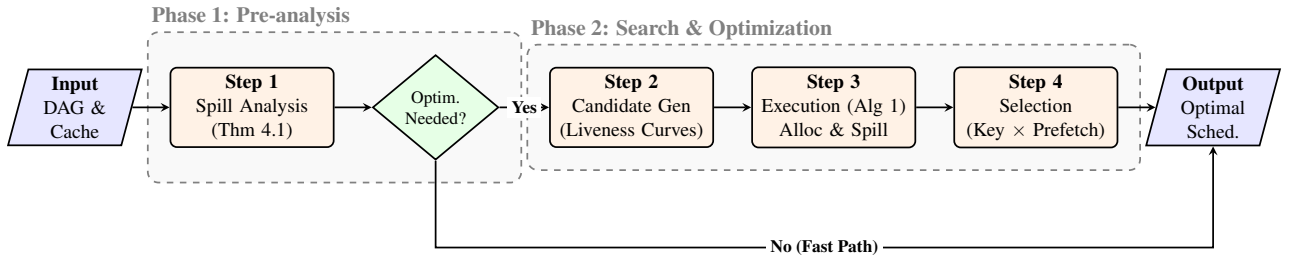


图 3: 方法总览。本文的优化框架由三个阶段组成：(1) 溢出不可避免性判定确定是否需要优化；(2) 三个互补的候选拓扑序通过活跃度塑形塑造不同的驻留曲线；(3) Algorithm 1 沿每个序执行地址分配与溢出插入，跨候选序与预取窗口的 portfolio 选优得到最终方案。

多 clean 储备的调度序会在三个层面同时获益。

## 4 方法

现有调度方法在容量超限后将溢出视为局部后果，在固定序上应用驱逐策略。然而实验表明，固定序时不同驱逐规则的 extra 差异通常不超过 4%，而不同合法序之间的 extra 摆动可达一个数量级 (§5.3)。因此本文方法将调度序本身作为主要优化自由度，通过改变序来调控溢出窗口内的 clean/dirty 驻留构成。

如图 3 所示，本文方法包含三个环节，各解决一个递进的子问题。首先，溢出判定 (§4.1) 回答给定 DAG 溢出是否为所有近最优调度都不可避免——若不可避免则将优化目标从消除溢出转为降低溢出代价，并建立溢出面积作为跨阶段代理目标。其次，候选序生成与地址分配 (§4.2) 以三种互补策略进行活跃度塑形，并沿序执行 best-fit 地址分配与代价感知的溢出插入 (Algorithm 1)。最后，将候选序与预取窗口的笛卡尔积逐一模拟，按词典键取最优组合。

### 4.1 溢出判定与代理目标

若存在某调度序使所有 cache 峰值驻留不超过容量，则无需溢出，clean/dirty 区分无关紧要。方法的第一步判断：对于给定 DAG，溢出是否为所有合理调度都无法避免的？为此建立线性时间可计算的判定证书。设  $\mathcal{O}_\epsilon$  为 extra 在最优值  $(1+\epsilon)$  倍以内的调度子集（ $\epsilon=0.1$ ）， $\underline{W}_c^\epsilon = \min_{S \in \mathcal{O}_\epsilon} \max_t W_c^S(t)$  为该子集上的工作集下界。

**定理 1** (溢出不可避免性证书). 若  $\underline{W}_c^\epsilon > \text{Cap}_c$ ，则  $\mathcal{O}_\epsilon$  中每一个调度都必须在 cache  $c$  上溢出，且对任意可行实现  $P$ ， $\text{extra}(P) \geq \max_t (W_c^S(t) - \text{Cap}_c)_+$ 。证明见补充材料 §2.2。

该证书刻意采用单侧条件： $\underline{W}_c^\epsilon > \text{Cap}_c$  确认溢出不可避免，但  $\underline{W}_c^\epsilon \leq \text{Cap}_c$  并不保证无溢出。本文将其作为适用域诊断，而非完整的无溢出可行性判定。一旦确认溢出不可避免，优化目标从消除溢出转为降低溢

出代价。

逐 cache 峰值是比较候选序溢出代价的直接选择，但 P1 最优序可能在某些 cache 上远超容量，导致 P2 代价反而更高。为此引入溢出面积作为代理目标：

$$A(S) = \sum_t \sum_{c \in \mathcal{C}} (\text{live}_c(t) - \text{Cap}_c)_+, \quad (1)$$

该指标同时度量超容量驻留的体积与持续时长。实现中使用归一化版本  $\Phi(S) = \sum_t \sum_c (\text{live}_c(t) - \text{Cap}_c)_+ / \text{Cap}_c$ 。下述定理在有界缺席时长假设下建立  $A(S)$  与最优溢出流量的双边常数关系：

**定理 2** (有条件溢出面积—extra 近似). 设每个 *spill* 缺席时长  $\leq L$ , 存在可行 *plan* 其移除面积  $R(P) \leq \lambda A(S)$  且每段缺席时长  $\geq \ell$ , 单位代价  $\kappa \in [1, 2]$ 。则

$$\frac{1}{L} A(S) \leq E^*(S) \leq \frac{2\lambda}{\ell} A(S). \quad (2)$$

证明与反例构造见补充材料 §2.4。

$\Phi$  兼具三项优势： $O(N)$  计算代价使候选序全量评估可行，跨 P1/P2/P3 一致的标量避免多目标口径切换，Theorem 2 解释了其为何在容量受限区域能够跟踪溢出流量。该代理目标不是对调度选择全局最优性的保证。

## 4.2 候选序生成与地址分配

候选序的设计原则为结构无关：所有序仅依赖缓冲的 clean/dirty 属性、大小和 cache 容量，不含算子类型的专用判断。三个候选序共享同一个“释放 → 操作 → 分配”三级就绪集的表调度结构：维护释放、操作、分配三级就绪集，释放节点优先调度以减少驻留压力，分配节点在就绪的释放和操作节点全部放行后才放行，从而压缩分配—释放间距。三级就绪集内的选择规则在各序间不同：压力感知序与容量节流序在各级内采用内存感知优先级，标识保留序在各级内一律按最小标识选择。具体采用三个互补的候选序：

- (1) 压力感知序：分配节点按后继入度排序——入度越低说明消费者越快就绪，延迟到即将被消费时放行，将分配时机压到峰值最不敏感处；该序同时作为 P1 的输出。
- (2) 容量节流序：分配节点按最小标识排序，并叠加容量门控——会导致 cache 超容的分配被暂时跳过，直到无法继续推迟时才强制放行最快解锁后续工作的分配；容量权重按  $1/\text{Cap}_c$  归一化以平衡不同 cache 的稀缺性。
- (3) 标识保留序：三级就绪集内各取最小标识，不含内存感知优先级与容量节流；其作用是保留稳定的输入缓冲顺序，使 COPY\_IN clean 缓冲更容易在计算窗口内常驻，从而在溢出窗口内留下可半价换出的 clean 储备。它是方法候选而非外部基线。

给定调度序，Algorithm 1 沿序执行地址分配与溢出插入。容量不足时按下式选择驱逐对象，将 §3 中定义的非对称代价编码进评分：

$$\text{victim} = \arg \max_b d(b) \cdot \frac{s_b}{e_b} = \arg \max_b \begin{cases} d(b), & b \text{ clean}, \\ d(b)/2, & b \text{ dirty}, \end{cases} \quad (3)$$

其中  $d(b)$  为下一次使用距离。dirty 缓冲的有效距离折半，使引擎优先驱逐廉价的 clean 缓冲。下述命题表明，在距离主导条件下，不同代价加权规则选出相同的驱逐集合：

**命题 1** (Belady-Margin 稳定性). 设距离主导的评分规则中代价比  $\rho = g_{\max}/g_{\min} \leq 2$ 。若被选驱逐集合的最小 *next-use* 距离超过未选集合最大距离的  $\rho$  倍，则所有此类规则选出相同的驱逐集合。这一策略的最优性论证见补充材料 §2.3。

---

**Algorithm 1** 地址分配与溢出插入

---

**Require:** 拓扑序  $S$ , cache 容量  $\{\text{Cap}_c\}$ , 预取窗口  $H$

**Ensure:** 扩展序 (含溢出节点)、地址分配、溢出列表

```
1: 初始化各 cache  $c$  的 best-fit 空闲区间管理器  $F_c$ 
2: for  $(t, v) \in \text{enumerate}(S)$  do
3:   if  $v = \text{Alloc}(b)$  then
4:      $\text{offset} \leftarrow \text{Place}(b, t)$ 
5:      $\text{memory}[b] \leftarrow \text{offset}$ 
6:   else if  $v = \text{Free}(b)$  then
7:     if  $b$  已被溢出 then  $\text{Reload}(b, t)$ 
8:     end if
9:      $F_{\tau(b)}.\text{Release}(\text{offset}_b, s_b)$ 
10:  else ▷ 操作节点
11:    for  $b \in v.\text{bufs}$ ,  $b$  已被溢出 do
12:       $\text{Reload}(b, t)$  ▷ 按需重载
13:    end for
14:    for  $b \in \text{SpilledSet}$  do ▷ 预取扫描
15:      if  $d(b) - t \leq H$  且  $F_{\tau(b)}$  有  $\geq s_b$  的空闲 then
16:         $\text{Reload}(b, t)$  ▷ 不触发驱逐
17:      end if
18:    end for
19:  end if
20: end for

21: procedure  $\text{Place}(b, t)$ 
22:    $\text{offset} \leftarrow F_{\tau(b)}.\text{BestFit}(s_b)$ 
23:   while  $\text{offset} = \text{nil}$  do ▷ 驱逐循环
24:      $v^* \leftarrow \arg \max_{b' \in \text{Resident}_{\tau(b)}} d(b') \cdot s_{b'} / e_{b'}$  ▷ 公式 (3)
25:     输出  $\text{SpillOut}(v^*)$ ;  $F_{\tau(b)}.\text{Release}(\text{offset}_{v^*}, s_{v^*})$ 
26:      $\text{offset} \leftarrow F_{\tau(b)}.\text{BestFit}(s_b)$ 
27:   end while
28:    $F_{\tau(b)}.\text{Allocate}(\text{offset}, s_b)$ 
29:   return  $\text{offset}$ 
30: end procedure
```

---

因此调度序为主要优化自由度：不同序之间 extra 摆动可达一个数量级，而固定序时不同驱逐规则的差异通常可忽略。

溢出缓冲的重载时机不影响溢出流量，但影响重载阻塞能否与计算重叠。Algorithm 1 中的预取窗口  $H$  控制该维度：已换出缓冲若在  $\leq H$  步内被使用且 cache 有空闲，则提前取回而不引发额外驱逐。P2 以最小化流量为主，用紧网格  $H \in \{0, 5, 40\}$ ；P3 以最小化时间为主，用宽网格  $H \in \{0, 5, 40, 80, 120\}$ 。将 3 个候选序与预取窗口网格做笛卡尔积，对每个组合执行 Algorithm 1，P2 按  $(E, n, T)$ 、P3 按  $(T, E, n)$  词典键取最优。候选集扩展具有单调性：加入新候选只改善或持平（见补充材料 §2.5）。

**复杂度分析。** 整体方法的时间复杂度为  $O(K \cdot W \cdot (N \log N + B^2))$ ，其中  $K = 3$  为候选序数， $W$  为预取窗口网格大小（P2 取 3，P3 取 5）， $N$  为 DAG 节点数， $B$  为缓冲数。候选序生成阶段的复杂度为  $O(N \log N)$ （拓扑排序 + 优先级队列维护）；地址分配阶段的瓶颈在 Belady 驱逐选择，最坏情况下每次分配扫描所有驻留缓冲，单次分配  $O(B)$ ，共  $B$  次分配，故  $O(B^2)$ 。流水模拟为  $O(N + E)$ ，其中  $E$  为 DAG 边数；在本文基准的稀疏 DAG 上，该项低于组合搜索和溢出分配项。

## 5 实验

本节围绕论文的核心问题展开实验验证：当容量溢出不可避免时，调度序能否通过改变峰值窗口内的 clean/dirty 驻留构成来降低真实溢出代价。实验从四个递进层面构建证据：公开 NPU 风格基准实例上的受控换序对照、机制分解与适用域分析、精确 oracle 对理论界的验证、以及 clean/dirty 代价差异的受控消融，以避免将结论建立在单一黑盒比较上。

### 5.1 实验设置

实验在一组公开的 NPU 核内调度实例上进行，这些实例由编译器风格的微操作 DAG 构成，覆盖卷积、注意力（FlashAttention）与矩阵乘三类主流算子，图规模从约  $1.7 \times 10^3$  到  $3.6 \times 10^4$  个节点不等，足以体现核函数级调度在容量受限下的典型行为（规模统计见表 1）。硬件抽象为 5 类片上 cache，其抽象容量分别为 L1=4,096、UB=1,024、L0A=256、L0B=256、L0C=512。所有实例仅用于评估，本文方法不依赖任何针对特定算子结构的硬编码。

表 1: 基准实例概况（FA = FlashAttention）。

| 实例           | 算子类型 | $ V $  | $ E $  | 缓冲数    |
|--------------|------|--------|--------|--------|
| Conv_Case0   | 卷积   | 2,580  | 3,869  | 831    |
| Conv_Case1   | 卷积   | 36,086 | 85,653 | 12,013 |
| FA_Case0     | 注意力  | 1,716  | 2,712  | 572    |
| FA_Case1     | 注意力  | 6,952  | 11,184 | 2,328  |
| Matmul_Case0 | 矩阵乘  | 4,160  | 7,104  | 1,216  |
| Matmul_Case1 | 矩阵乘  | 30,976 | 55,040 | 8,960  |

为避免结论建立在单一黑盒比较之上，评估沿四个递进层面组织证据（表 2）：公开实例上的同引擎换序对照、合成分布上的适用域分析、小图上的精确 oracle 验证，以及隔离 clean/dirty 代价机制的受控消融。其核心是一致的同引擎口径：所有对照方法与本文方法共享同一 best-fit 地址分配与溢出引擎（Algorithm 1），仅替换输入的拓扑序，从而将溢出流量的差异干净地归因于序对 clean/dirty 驻留构成的塑形，而非驱逐策略的实现差异。评估指标沿 §3 的三个视角展开——P1 度量逐 cache 峰值驻留，P2 度量总溢出流量  $E(S)$ ，P3 度量流水化执行时间——其中 P2 直接刻画本文关注的溢出代价，故作为主要报告口径。

表 4: P2 溢出流量  $E(S)$  对照（同引擎口径）。FA = FlashAttention。最优值加粗。

| 实例       | cp_list   | pressure  | g_hsu     | cp_free        | 本文             |
|----------|-----------|-----------|-----------|----------------|----------------|
| Conv_0   | 694,506   | 207,932   | 210,844   | 212,956        | <b>88,044</b>  |
| Conv_1   | 1,695,264 | 1,048,524 | 1,053,484 | 73,348         | <b>72,520</b>  |
| FA_0     | 211,784   | 101,356   | 90,092    | <b>3,904</b>   | <b>3,904</b>   |
| FA_1     | 965,944   | 445,876   | 418,228   | 33,152         | <b>32,920</b>  |
| Matmul_0 | 823,168   | 296,064   | 298,240   | 34,944         | <b>34,688</b>  |
| Matmul_1 | 6,506,240 | 2,556,928 | 2,560,640 | <b>460,800</b> | <b>460,800</b> |

表 2: 实验中的四层证据。

| 层级             | 规模       | 作用                                        |
|----------------|----------|-------------------------------------------|
| 公开 NPU case    | 6 个 DAG  | 同引擎换序对照，检验编译器风格实例上的流量差异。                  |
| 合成分布           | 36 个 DAG | 区分 capacity-bound 与 order-reachable 两类区域。 |
| CP-SAT oracle  | 8 个小图    | 验证给定序下的最优性与理论界。                           |
| clean/dirty 消融 | GEMM 变体  | 隔离 $2\times$ 储备代价机制。                      |

对照方法取自经典的延迟导向与内存压力导向范式，且均对 clean/dirty 区分无感。关键路径表调度 [9, 15] 以最小化关键路径延迟为目标而不考虑内存压力；均匀压力贪心最小化逐步活跃字节峰值，但将 clean 与 dirty 缓冲等同看待；Goodman-Hsu 风格的延迟/压力切换 [6] 代表经典的 integrated-prepass 思路，在关键路径优先与压力降低之间动态切换。在此之外，本文引入 free-first critical-path companion 作为一个更强的对照——它在关键路径序上优先调度释放节点以缩短活跃区间，对分配器更友好但仍不含显式的代价感知，用以检验“提前释放、延后分配”本身能在多大程度上解释本文方法的收益。

在效率上，本文求解器对最大实例仍保持实用的运行开销。P1 仅生成单个调度序，所有实例均在 0.2 秒内完成；P2、P3 因需在候选序与预取窗口的笛卡尔积上模拟，开销随实例规模增长，最大实例 Conv\_Case1 的 P3 用时约 73 秒（表 3，3 次重复中位数）。

表 3: 求解器 wall-clock 运行时间（秒，3 次重复中位数）。

| 实例       | P1    | P2     | P3     |
|----------|-------|--------|--------|
| Conv_0   | 0.009 | 0.391  | 0.683  |
| Conv_1   | 0.189 | 57.971 | 73.141 |
| FA_0     | 0.006 | 0.220  | 0.328  |
| FA_1     | 0.027 | 2.723  | 3.356  |
| Matmul_0 | 0.014 | 0.701  | 1.137  |
| Matmul_1 | 0.126 | 19.402 | 32.328 |

## 5.2 主对照结果

图 4 和表 4 给出 6 个公开基准 case 上各调度序在同一引擎下的 P2 溢出流量（对数轴）。内存压力感知但 clean/dirty 无感的均匀压力贪心与 Goodman-Hsu 混合调度，在所有 case 上均显著劣于本文方法，额外付出  $2.4\text{--}26\times$  的溢出流量（中位约  $11\times$ ）。纯延迟导向的关键路径表调度差距更大，约  $8\text{--}54\times$ 。这一系统性差距表明，仅压低总活跃字节峰值或关键路径延迟不足以逼近真实的溢出代价下界；关键在于把峰值窗口中的可驱逐储备塑形成更便宜的 clean 字节。在全部 6 个 case、3 个视角、4 个对照方法共 72 个组合中，本文序均取得了更低或持平的评估目标。

值得注意的是，free-first critical-path companion 是一个具有竞争力的强基线：除 Conv\_Case0（落后  $2.4\times$ ）外，其 P2 溢出流量与本文方法的中位差距仅约 1%，在 FA\_Case0 和 Matmul\_Case1 上完全持平。这



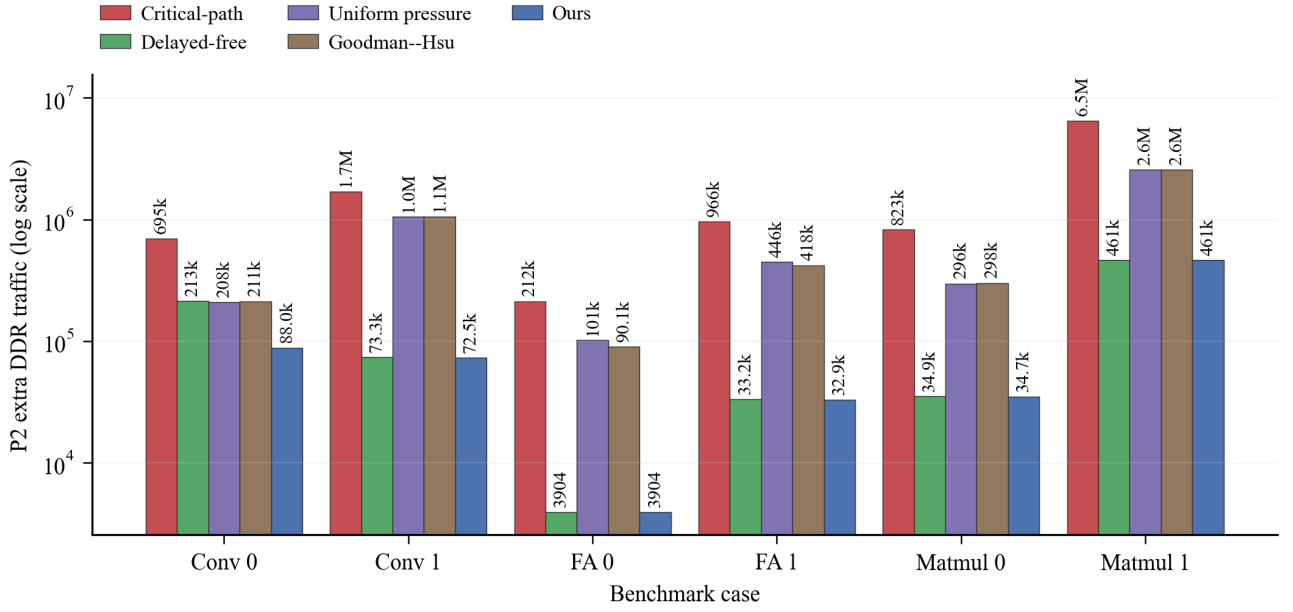


图 4: 标准调度器对照 (P2 溢出流量, 对数轴)。所有方法共享同一地址分配/溢出引擎, 仅替换调度序。

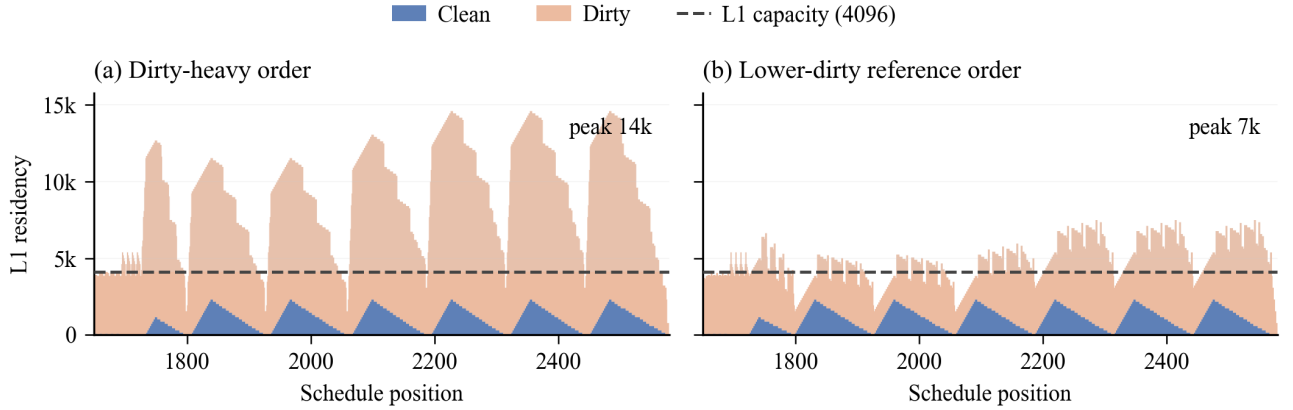


图 5: 基准 case 上的 clean/dirty 驻留分解 (仅显示 L1 高压窗口)。蓝色为已有片外备份的 clean 驻留, 橙色为需写回的 dirty 驻留, 虚线为 L1 容量。同一容量约束下, 不同调度序暴露不同的峰值高度与 dirty 构成。

表明对分配器友好的延迟序已能通过缩短活跃区间间接减少溢出, 捕获一部分收益。然而, 该基线缺乏显式的 clean/dirty 代价塑形, 因此在 Conv\_Case0 等 clean 储备对代价有决定性影响的实例上仍被系统性拉开差距, 且在多目标评估中一致被本文方法压制。

### 5.3 机制验证与适用域分析

上节的对照实验表明溢出流量存在系统性差距, 本节进一步追问收益的来源与适用边界。

**调度序与驱逐策略的边际效应** 固定调度序时, 4 种 Belady 风格驱逐变体的 extra 在 Matmul 与 FA 实例上完全相同 (变异系数  $CV = 0$ ), Conv 行 CV 最高约 4%; 而同一引擎下不同序的 extra 摆动可达  $10\times$  以上。图 5 将一个基准 case 的 L1 高压窗口分解为 clean 与 dirty 驻留: 两个合法序在相同容量约束下, 暴露出显著不同的 dirty 占比与峰值高度。这一结果印证了 Proposition 1 的理论预测: 在距离主导条件下驱逐策略的边际收益远小于调度序, 后者才是影响溢出代价的主要自由度。

**适用域** 合成 DAG 分布上的系统评测进一步界定了方法的适用边界。在 Theorem 1 证书标记为 capacity-bound 的区域, 本文方法相对关键路径表调度和随机序的胜率均为 100%, 相对 free-first companion 的胜率



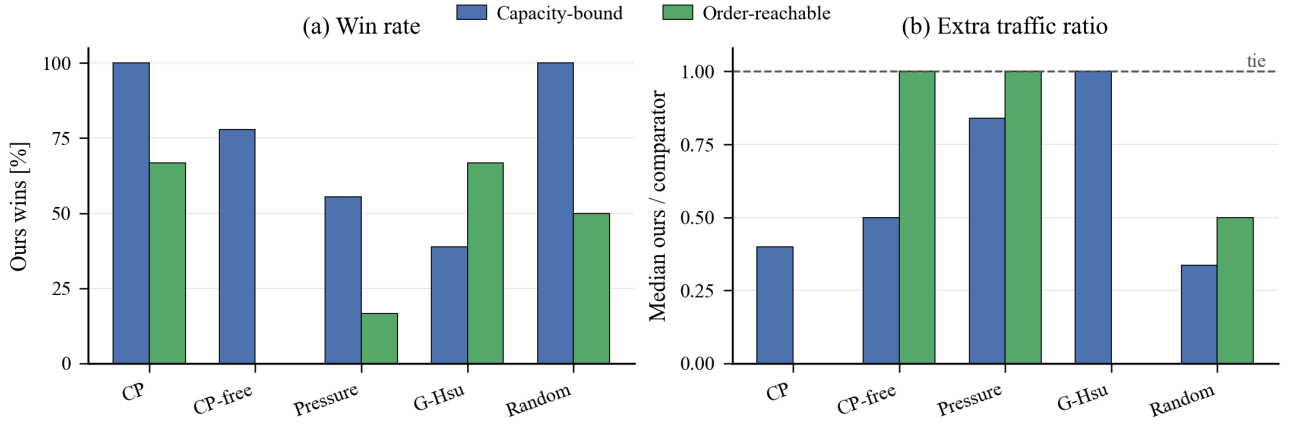


图 6: 合成分布上的适用域。左图为本文方法相对各 comparator 的胜率；右图为 extra 中位比值 (ours/comparator, 越低越好), 虚线表示持平。本文方法在 capacity-bound 区域优势最明显, 而在好序可直接避免溢出的 order-reachable 区域自然失去系统优势。

为 77.8% (中位  $2\times$  优势)。而在 order-reachable 子群中, 好序可直接避免溢出, clean/dirty 塑形无从发挥, 本文方法相对 free-first companion 的胜率降至 0%。这一分布与理论证书高度一致: 证书标记的是“必须优化溢出代价构成”的区域, 方法的有效性严格对应于这一前提条件。

**理论验证与代理目标** 小图 CP-SAT oracle 验证了两项结论: 给定所选序, 本文引擎达到 capacity-only 最优 extra (比值为 1.0); Theorem 1 的下界在全部小图成立, Theorem 2 的经验比  $E^*/A \in [0.56, 1.13]$  落在常数因子带内。溢出面积代理  $\Phi$  与逐 cache 峰值的全局 Spearman 相关性几乎相同 (0.958 vs 0.955);  $\Phi$  的价值不在于显著提高预测精度, 而在于  $O(N)$  计算代价使候选序全量评估可行, 且 Theorem 2 为其提供跨阶段一致的理论支撑。详图与逐实例数据见补充材料。

**受控消融** 为排除样例特异性, 本文构建合成 GEMM 核并固定同一 DAG 结构、相同 spill 数和相同峰值, 仅改变峰值窗口中的储备类型。clean 储备的 extra 为 1,536, dirty 储备为 3,072, 恰为  $2\times$ , 在完全隔离的条件下印证了 clean 与 dirty 缓冲间  $2\times$  代价非对称的理论预测。进一步扫描调度序可见, extra 随峰值处 clean 占比的上升而单调下降。

## 5.4 局限性

本文方法的适用边界需要明确界定。若实例处于 order-reachable 区域, 即存在某个合法序可使所有 cache 峰值驻留不超过容量, 则无需溢出, clean/dirty 塑形无从发挥; 若某个传统压力/延迟混合序已保留了足够的 clean 储备, 本文方法的优势也会缩小甚至消失。此外, Theorem 2 的近似解释依赖有界缺席时长假设, 长寿命小溢出的场景可能削弱代理目标的有效性。本文亦仅针对编译期完全已知的单核静态 DAG, 多核、动态控制流与更复杂存储层次下的推广留待后续工作。

## 6 结论

本文揭示了 NPU 核内调度中的一个结构性杠杆: clean/dirty 溢出代价的  $2\times$  非对称, 使调度序不仅影响活跃字节峰值, 还能主动塑形容容量峰值窗口中的驻留构成。当溢出不可避免时, 保留更多可廉价换出的 clean 储备, 可以显著降低真实溢出流量。围绕这一发现, 本文给出了溢出不可避免性证书 (Theorem 1)、有条件溢出面积近似定理 (Theorem 2), 并用 Belady-margin 稳定性 (Proposition 1) 解释了“驱逐策略相对不敏感、调度序高度敏感”的经验现象。

实验从公开 NPU 风格基准 case、合成分布、小图 oracle 和受控消融四个层面支持这一结论。在公开基准 case 上, clean/dirty 无感的标准内存压力调度器比本文方法多付  $2.4\text{--}26\times$  的 P2 溢出流量 (§5.2)。合

成分分布表明，收益集中在证书所界定的 capacity-bound 区域；在 order-reachable 区域，简单序已经可以避免溢出，本文方法没有系统优势 (§5.3)。小图 CP-SAT oracle 进一步验证了给定序下的引擎最优性，以及两个理论界的经验常数范围 (§5.3)。受控消融则隔离出 clean 与 dirty 储备的  $2\times$  代价差异，说明收益并非来自样例偶然性 (§5.3)。

未来工作包括：将溢出代价感知的活跃度塑形推广到多核调度与跨核缓存一致性场景；支持含动态控制流（条件分支、循环）的计算图；以及对更复杂存储层级中跨层溢出级联效应的建模。

## 参考文献

- [1] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2): 78–101, 1966.
- [2] Prasanth Chatarasi, Alex Gatea, Wei Wang, Chris Bowler, Shubham Jain, Masoud Ataei Jaliseh, Nicole Khoun, Alberto Mannari, Bardia Mahjour, Viji Srinivasan, and Swagath Venkataramani. Enabling spill-free compilation via affine-based live range reduction optimization. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 1–13, 2026.
- [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint*, abs/1604.06174, 2016.
- [4] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594, Carlsbad, CA, 2018. USENIX Association. URL <https://www.usenix.org/conference/osdi18/presentation/chen>.
- [5] Christian Coester, Roie Levin, Joseph (Seffi) Naor, and Ohad Talmon. Competitive algorithms for block-aware caching. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 2022.
- [6] James R. Goodman and Wei-Chung Hsu. Code scheduling and register allocation in large basic blocks. In *Proceedings of the 2nd International Conference on Supercomputing (ICS)*, pages 442–452, 1988.
- [7] Jia Guo, Maria Jesus Garzaran, and David Padua. The power of Belady’s algorithm in register allocation for long basic blocks. In *Languages and Compilers for Parallel Computing (LCPC)*, volume 2958 of *Lecture Notes in Computer Science*, pages 374–389. Springer, 2004. doi: 10.1007/978-3-540-24644-2\_24.
- [8] Paras Jain, Ajay N. Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. Checkmate: Breaking the memory wall with optimal tensor rematerialization. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020.
- [9] Yu-Kwong Kwok and Ishfaq Ahmad. Static scheduling algorithms for allocating directed task graphs to multiprocessors. *ACM Computing Surveys*, 31(4):406–471, 1999.
- [10] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 2–14, 2021. doi: 10.1109/CGO51591.2021.9370308.
- [11] Yi Li, Aarti Gupta, and Sharad Malik. Combined scheduling, memory allocation and tensor replacement for minimizing off-chip data accesses of DNN accelerators. *arXiv preprint*, abs/2311.18246, 2023.

- [12] Vincenzo Liberatore, Martin Farach-Colton, and Ulrich Kremer. Evaluation of algorithms for local register allocation. In *Proceedings of the 8th International Conference on Compiler Construction (CC)*, volume 1575 of *Lecture Notes in Computer Science*, pages 137–153. Springer, 1999. doi: 10.1007/978-3-540-49051-7\_10.
- [13] Ghassan Shobaki, Saeed Shawabkeh, and Najm Eldeen Abu-Rmaileh. Preallocation instruction scheduling with register pressure minimization using a combinatorial optimization approach. *ACM Transactions on Architecture and Code Optimization*, 10(4), 2013.
- [14] Ghassan Shobaki, Ahmad Al Qutami, and Najm Eldeen Abu-Rmaileh. Register-pressure-aware instruction scheduling using ant colony optimization. *ACM Transactions on Architecture and Code Optimization*, 19(4), 2022.
- [15] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3):260–274, 2002. doi: 10.1109/71.993206.